

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2– Interactive Tree Simulator

Course Code:	CSA0316	Course Title:	Data Structures
CO Assessed:	CO3 – Imitation (BL3, BL4)	Assessment:	Assessment Tool 2 – Interactive Tree Simulator
Weightage:	20 % of CIA	Total Marks:	100
CO3 Statement:	Solve problems for optimized data storage and retrieval using Binary Search Trees, AVL Trees, B-Trees, Red-Black Trees, and Splay Trees		
Student Name:	G. Hima Bindu	Reg. No.:	192511377
Section / Batch:	B.Tech CSE	Date:	17-08-26

Code of Conduct

I (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

Rubrics

S. No.	Rubric	Weightage	Excellent (4)	Good (3)	Satisfactory (2)	Needs Improvement (1)
1	Tree Operations & Algorithm Correctness	25%	All operations work correctly and algorithms are accurate	Minor errors in some operations	Several operations have errors	Major operations are missing/incorrect
2	Interactive Visualization	25%	Tree changes dynamically and clearly with each operation	Good visualization with minor issues	Basic visualization with limited interaction	Visualization is unclear/non-functional
3	User Interface & Usability	20%	Highly intuitive, clear, and easy to use	User-friendly with minor issues	Usable but needs improvement	Difficult to navigate/use
4	Educational Value & Explanation	15%	Clearly explains tree concepts, operations, and thiiiraversals	Good explanation with minor gaps	Basic explanation	Explanation is inadequate
5	Innovation & Code Quality	15%	Creative features with clean, modular, well-documented code	Good implementation and some creativity	Basic implementation	Poor code quality and little innovation

List of Trees

- Binary Tree
- Binary Search Tree
- AVL Tree
- Red Black Tree
- Trie Tree
- B Tree
- B+ Tree
- Splay Tree
- Quad Tree

B-TREE STIMULATION

INSERTION:

Visualization of B-Tree

CSA0316 Data Structures

Team Members

1. M Kyathi Sai Priya
2. G Himabindu
3. Shaik Ayesha Tabassum

Operations

Insertion
Add a new key

Deletion
Remove a key

Searching
Find a key

Use the buttons below to perform each operation in real time and observe the B-Tree after every step.

Real-Time B-Tree Simulator

Order (max children) Enter key

Insert

Delete

Searching

Random Insert

Clear

Order m = 4 → Maximum children = 4, Maximum keys = 3, Minimum keys in non-root node = 1.

9 already exists.

B-Tree Visualization

```

graph TD
    9[9] --> 7[7]
    9 --> 29_70["29 | 70"]
    7 --> 6[6]
    7 --> 8[8]
    29_70 --> 18_25["18 | 25"]
    29_70 --> 49[49]
    29_70 --> 78_85_85["78 | 85 | 85"]
  
```

Keys
12

Height
3

Nodes
8

Order
4

Visualization of B-Tree

CSA0316 Data Structures

Team Members

1. M.Kyathi Sai Priya
2. G.Himabindu
3. Shaik Ayesha Tabassum

Definition of B-Tree

A B-Tree is a self-balancing multiway search tree in which each node can contain multiple sorted keys and multiple children. It keeps all leaves at the same level and is widely used for efficient searching, insertion and deletion.

Properties of B-Tree

- A node can have at most m children and m - 1 keys.
- Keys in every node are stored in sorted order.
- All leaf nodes are at the same level.
- The tree remains balanced after insertion and deletion.
- Overflow causes splitting; underflow may cause borrowing or merging.

Operations

Insertion

Add a new key

Deletion

Remove a key

Searching

Find a key

Use the buttons below to perform each operation in real time and observe the B-Tree after every step.

Real-Time B-Tree Simulator

Order (max children)

4

Enter key

10

Insert

Delete

Searching

Random Insert

Clear

Order m = 4 → Maximum children = 4, Maximum keys = 3, Minimum keys in non-root node = 1.

Inserted 10.

B-Tree Visualization

```

graph TD
    Root["9"] --> L["7"]
    Root --> R["29 | 70"]
    L --> L1["6"]
    L --> L2["8"]
    R --> R1["10 | 16 | 26"]
    R --> R2["49"]
    R --> R3["78 | 85 | 86"]
    
```

Keys

13

Height

3

Nodes

8

Order

4

DELETION:

Visualization of B-Tree

CSA0316 Data Structures

Team Members

1. M.Kyathi Sai Priya
2. G.Himabindu
3. Shaik Ayesha Tabassum

Definition of B-Tree

A B-Tree is a self-balancing multiway search tree in which each node can contain multiple sorted keys and multiple children. It keeps all leaves at the same level and is widely used for efficient searching, insertion and deletion.

Properties of B-Tree

- A node can have at most m children and m - 1 keys.
- Keys in every node are stored in sorted order.
- All leaf nodes are at the same level.
- The tree remains balanced after insertion and deletion.
- Overflow causes splitting; underflow may cause borrowing or merging.

Operations

Insertion

Add a new key

Deletion

Remove a key

Searching

Find a key

Use the buttons below to perform each operation in real time and observe the B-Tree after every step.

Real-Time B-Tree Simulator

Order (max children)

4

Enter key

10

Insert

Delete

Searching

Random Insert

Clear

Order m = 4 → Maximum children = 4, Maximum keys = 3, Minimum keys in non-root node = 1.

Inserted 10.

B-Tree Visualization

```

graph TD
    Root["9"] --> L["7"]
    Root --> R["29 | 70"]
    L --> L1["6"]
    L --> L2["8"]
    R --> R1["10 | 16 | 26"]
    R --> R2["49"]
    R --> R3["78 | 85 | 86"]
    
```

Keys

13

Height

3

Nodes

8

Order

4

Visualization of B-Tree

CSA0316 Data Structures

Team Members

1. M Kyathi Sai Priya
2. G Himabindu
3. Shaik Ayesha Tabassum

Definition of B-Tree

A B-Tree is a self-balancing multiway search tree in which each node can contain multiple sorted keys and multiple children. It keeps all leaves at the same level and is widely used for efficient searching, insertion and deletion.

Properties of B-Tree

- A node can have at most m children and m - 1 keys.
- Keys in every node are stored in sorted order.
- All leaf nodes are at the same level.
- The tree remains balanced after insertion and deletion.
- Overflow causes splitting; underflow may cause borrowing or merging.

Operations

Insertion
Add a new key

Deletion
Remove a key

Searching
Find a key

Use the buttons below to perform each operation in real time and observe the B-Tree after every step.

Real-Time B-Tree Simulator

Order (max children) Enter key

4 18

Insert Delete Searching Random Insert Clear

Order m = 4 → Maximum children = 4, Maximum keys = 3, Minimum keys in non-root node = 1.

Deleted 18.

B-Tree Visualization

```
graph TD; 9[9] --> 7[7]; 9 --> 29_70["29 | 70"]; 7 --> 6[6]; 7 --> 8[8]; 29_70 --> 10_26["10 | 26"]; 29_70 --> 49[49]; 29_70 --> 78_85_85["78 | 85 | 85"]
```

Keys 12 Height 3 Nodes 8 Order 4

SEARCHING:

Visualization of B-Tree

CSA0316 Data Structures

Team Members

1. M Kyathi Sai Priya
2. G Himabindu
3. Shaik Ayesha Tabassum

Definition of B-Tree

A B-Tree is a self-balancing multiway search tree in which each node can contain multiple sorted keys and multiple children. It keeps all leaves at the same level and is widely used for efficient searching, insertion and deletion.

Properties of B-Tree

- A node can have at most m children and m - 1 keys.
- Keys in every node are stored in sorted order.
- All leaf nodes are at the same level.
- The tree remains balanced after insertion and deletion.
- Overflow causes splitting; underflow may cause borrowing or merging.

Operations

Insertion
Add a new key

Deletion
Remove a key

Searching
Find a key

Use the buttons below to perform each operation in real time and observe the B-Tree after every step.

Real-Time B-Tree Simulator

Order (max children) Enter key

4 28

Insert Delete Searching Random Insert Clear

Order m = 4 → Maximum children = 4, Maximum keys = 3, Minimum keys in non-root node = 1.

Deleted 18.

B-Tree Visualization

```
graph TD; 9[9] --> 7[7]; 9 --> 29_70["29 | 70"]; 7 --> 6[6]; 7 --> 8[8]; 29_70 --> 10_26["10 | 26"]; 29_70 --> 49[49]; 29_70 --> 78_85_85["78 | 85 | 85"]
```

Keys 12 Height 3 Nodes 8 Order 4

Visualization of B-Tree

CSA0316 Data Structures

Team Members

1. M.Kyathi Sai Priya
2. G.Himabindu
3. Shaik Ayesha Tabassum

Definition of B-Tree

A B-Tree is a self-balancing multiway search tree in which each node can contain multiple sorted keys and multiple children. It keeps all leaves at the same level and is widely used for efficient searching, insertion and deletion.

Properties of B-Tree

- A node can have at most m children and $m - 1$ keys.
- Keys in every node are stored in sorted order.
- All leaf nodes are at the same level.
- The tree remains balanced after insertion and deletion.
- Overflow causes splitting; underflow may cause borrowing or merging.

Operations

Insertion
Add a new key

Deletion
Remove a key

Searching
Find a key

Use the buttons below to perform each operation in real time and observe the B-Tree after every step.

Real-Time B-Tree Simulator

Order (max children) Enter key

4

25

Insert

Delete

Searching

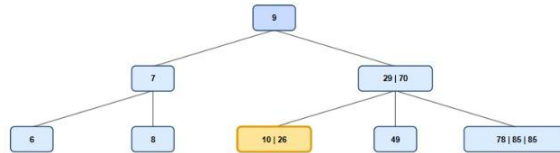
Random Insert

Clear

Order $m = 4 \rightarrow$ Maximum children = 4, Maximum keys = 3, Minimum keys in non-root node = 1.

26 is present in the B-Tree.

B-Tree Visualization



Keys
12

Height
3

Nodes
8

Order
4